

# **AN INTELLIGENT AGENT FOR FAILURE ANALYSIS USING HUMAN-IN-THE-LOOP LEARNING**

BITS ZG628T: Dissertation

By

**MOGALIPUVVU TEJA PRANAVI**

**2024DA04351**

Dissertation work carried out at

**Personal Laptop, Hyderabad**

Submitted in partial fulfilment of **M.Tech. Data Science  
& Engineering** degree programme

Under the Supervision of

**SRINIVAS REDDY VENKATRAM**

**PDI TECHNOLOGIES**



**BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE**

**PILANI (RAJASTHAN)**

**JUNE 2026**

## Contents

| S. No. | Title                                                 | Page Number |
|--------|-------------------------------------------------------|-------------|
| 1.     | <a href="#"><u>Abstract</u></a>                       | 3           |
| 2.     | <a href="#"><u>Research Gap</u></a>                   | 4 - 5       |
| 3.     | <a href="#"><u>Modules</u></a>                        | 6 - 10      |
| 4.     | <a href="#"><u>Functional Block Diagram</u></a>       | 11 -15      |
| 5.     | <a href="#"><u>Major Technical Specifications</u></a> | 16 - 17     |
| 6.     | <a href="#"><u>Design Considerations</u></a>          | 18          |
| 7.     | <a href="#"><u>Future Plan</u></a>                    | 19          |
| 8.     | <a href="#"><u>Abbreviations</u></a>                  | 20          |

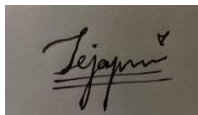
## 1. Abstract

Modern DevOps ecosystems depend on interconnected pipelines, infrastructure services, monitoring platforms, and automation workflows to sustain continuous software delivery. As these environments grow in complexity, operational failures become increasingly difficult to diagnose due to the large volume of logs, metrics, alerts, configuration changes, and historical incident records distributed across multiple systems. Conventional root cause analysis often relies on manual investigation, requiring significant expert effort and resulting in prolonged service disruptions and delayed recovery.

This dissertation presents **An Intelligent Agent for Failure Analysis Using Human-in-the-Loop Learning**, a multi-agent cognitive framework designed to support automated failure diagnosis and controlled remediation within DevOps environments. The proposed approach combines **Agentic Retrieval-Augmented Generation (RAG)**, **Large Language Models**, **autonomous reasoning agents**, and **Human-in-the-Loop governance** to create a continuously improving operational intelligence system. An Orchestrator Agent coordinates specialized agents responsible for root cause identification, infrastructure validation, remediation planning, and experiential learning. Contextual knowledge is retrieved from organizational repositories, historical incidents, operational documents, and monitoring data through an Onyx-based RAG layer, while HolmesGPT independently performs parallel failure investigation to provide an additional analytical perspective. The outputs from both reasoning paths are consolidated to improve diagnostic confidence and decision quality. Approved remediation actions are executed through controlled automation workflows, while human oversight mechanisms ensure compliance, transparency, and operational safety.

The proposed framework aims to reduce Mean Time to Resolution (MTTR), improve diagnostic accuracy, enhance operational resilience, and capture organizational knowledge from every resolved incident. By integrating intelligent reasoning, automated execution, and continuous feedback-driven learning, the research contributes a scalable model for next-generation AIOps systems capable of evolving alongside enterprise software delivery environments.

---

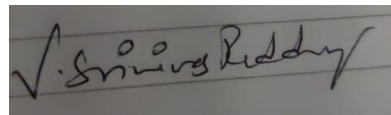


**Signature of the Student**

Name: M. Teja Pranavi

Date: 21-06-2026

Place: Hyderabad



**Signature of the Supervisor**

Name: Srinivas Venkatram

Date: 21-06-2026

Place: Hyderabad

## 2. Research Gap

Despite significant progress in Large Language Model (LLM)-based agents, AIOps platforms, and autonomous reliability engineering frameworks, several limitations remain unresolved in current research and industrial implementations.

### ➤ **Limited Integration of Multi-Source Operational Context**

Most existing studies focus on analyzing a single category of operational evidence, such as logs, alerts, metrics, or source code artifacts. There remains a lack of unified frameworks capable of simultaneously reasoning across pipeline executions, infrastructure telemetry, organizational knowledge repositories, historical incidents, and operational documentation to support context-rich failure diagnosis.

### ➤ **Insufficient Active Validation During Root Cause Analysis**

Current intelligent troubleshooting systems primarily rely on passive observation of available data. Few approaches allow agents to actively perform infrastructure validation tasks, such as verifying service availability, checking deployment states, validating cluster health, or inspecting runtime configurations before finalizing a diagnostic conclusion.

### ➤ **Absence of Collaborative Multi-Agent Decision Frameworks**

Although recent research explores autonomous agents, most solutions depend on a single reasoning pathway. Existing systems rarely employ multiple specialized agents that independently investigate incidents and collaboratively synthesize evidence to improve diagnostic confidence and reduce reasoning errors.

### ➤ **Lack of Cross-Verification Between Independent Diagnostic Engines**

State-of-the-art operational intelligence platforms generally generate root cause assessments through a single model or reasoning workflow. There is limited research on architectures that compare and reconcile findings from independent diagnostic mechanisms to produce more trustworthy and explainable outcomes.

### ➤ **Human Oversight Remains Weakly Integrated**

Many autonomous remediation frameworks prioritize full automation. However, enterprise environments require accountability, approval mechanisms, and risk-aware intervention before executing operational changes. Current literature provides limited support for structured Human-in-the-Loop governance throughout the diagnosis and remediation lifecycle.

### ➤ **Inadequate Compliance-Aware Automation**

Existing remediation systems often focus on technical feasibility rather than operational governance. There is a shortage of frameworks capable of enforcing policy-driven execution controls where actions are automatically classified as permitted, restricted, or approval-dependent based on organizational compliance requirements.

### ➤ **Limited Continuous Learning from Operational Experience**

Most current systems treat incident resolution as a terminal event. Human feedback, approved remediation actions, corrective modifications, and incident outcomes are rarely

captured and transformed into reusable organizational knowledge that can improve future troubleshooting performance.

➤ **Lack of End-to-End Agent Lifecycle Observability**

Traditional observability practices provide visibility into applications and infrastructure but offer limited insight into agent behavior. Research is still evolving regarding mechanisms for tracing agent reasoning paths, monitoring tool interactions, evaluating decision confidence, and measuring operational effectiveness during live incident handling.

➤ **Gap Between Experimental Research and Enterprise Deployment Reality**

A significant portion of current literature evaluates autonomous operational agents within controlled environments or limited-scale testbeds. Practical challenges encountered in enterprise ecosystems—including heterogeneous tooling, legacy infrastructure dependencies, fragmented data sources, security restrictions, and governance requirements—remain insufficiently addressed.

➤ **Absence of a Unified Cognitive AIOps Framework**

Current solutions typically address isolated stages of the incident management lifecycle, such as monitoring, diagnosis, or remediation. There is a lack of comprehensive architectures that seamlessly integrate knowledge retrieval, multi-agent reasoning, infrastructure validation, human supervision, controlled automation, and continuous learning into a single operational intelligence framework.

### 3. Modules

#### Module 1: DevOps Environment and Failure Simulation Module

This module is responsible for creating the controlled DevOps environment where failures can be generated and analysed. The environment consists of infrastructure components such as virtual machines, containers, Kubernetes workloads, CI/CD pipelines, configuration management tools, and monitoring systems. Different failure scenarios such as service crashes, deployment errors, configuration issues, and resource problems are simulated to generate operational incidents. The module provides realistic failure data that acts as input for the intelligent failure analysis system.

**Functionality:**

- Creates local DevOps infrastructure.
- Generates controlled failure scenarios.
- Collects operational logs and metrics.
- Provides incident inputs for analysis.

**Outcome:**

A realistic DevOps test environment where the proposed AI-based failure analysis framework can be evaluated.

#### Module 2: Observability and Data Collection Module

This module collects real-time operational information from different DevOps components. It integrates monitoring sources to capture logs, metrics, traces, alerts, and system events generated during failures. The collected telemetry data provides the evidence required by AI agents for understanding system behaviour. This module acts as the data foundation for root cause analysis.

**Functionality:**

- Collects logs from applications and infrastructure.
- Captures performance metrics and traces.
- Monitors service health and availability.
- Sends incident context to AI agents.

**Outcome:**

A centralized operational data stream containing the complete technical context of failures.

#### Module 3: Knowledge Repository and RAG Module

This module manages the organizational knowledge required for intelligent diagnosis. It stores previous incident reports, troubleshooting documents, runbooks, configuration guides, and resolution procedures. The RAG mechanism retrieves relevant historical knowledge whenever a new failure occurs. The retrieved information improves the reasoning capability of the AI agents by providing domain-specific context.

**Functionality:**

- Ingests documents and historical incidents.

- Creates semantic embeddings.
- Performs similarity-based information retrieval.
- Provides contextual knowledge to agents.

**Outcome:**

A knowledge-aware diagnosis process that reduces dependency on manual troubleshooting.

**Module 4: Intelligent Agent Orchestration Module**

This module acts as the central decision-making component of the system. The Orchestrator Agent coordinates communication between different specialized agents and assigns tasks based on the failure scenario. It manages the complete workflow from investigation to remediation recommendation. The module enables multiple AI agents to collaborate instead of depending on a single reasoning model.

**Functionality:**

- Receives incident information.
- Coordinates RCA, validation, and remediation agents.
- Manages agent communication.
- Combines outputs from multiple reasoning sources.

**Outcome:**

A collaborative multi-agent system capable of performing complex failure investigations.

**Module 5: Root Cause Analysis (RCA) Agent Module**

The RCA Agent performs intelligent investigation of detected failures using retrieved knowledge and operational data. It analyses logs, metrics, historical incidents, and system information to identify possible causes. The agent uses Large Language Models with RAG-based context to generate evidence-based explanations. It provides the initial root cause hypothesis for further validation.

**Functionality:**

- Understands incident description.
- Retrieves similar past failures.
- Analyses available evidence.
- Generates root cause possibilities.

**Outcome:**

A detailed RCA report containing probable failure reasons and supporting evidence.

**Module 6: Infrastructure Validation Agent Module**

This module validates the RCA generated by the reasoning agents by interacting with the actual DevOps environment. It performs health checks and verifies whether the suspected cause exists in the infrastructure. The agent communicates with monitoring tools, Kubernetes resources, pipelines, and configuration systems. This reduces incorrect AI-generated conclusions.

**Functionality:**

- Checks service status.
- Validates infrastructure conditions.
- Confirms configuration states.
- Provides verification evidence.

**Outcome:**

A validated failure diagnosis with higher accuracy and reliability.

**Module 7: HolmesGPT Independent Analysis Module**

This module introduces an independent AI investigation path for improving diagnostic confidence. HolmesGPT analyses the incident separately using available operational information and produces its own root cause assessment. The independent analysis is compared with the primary RCA Agent output. This dual-analysis approach reduces dependency on a single AI reasoning process.

**Functionality:**

- Performs independent failure investigation.
- Analyses logs and system behaviour.
- Generates alternative RCA.
- Provides confidence comparison.

**Outcome:**

A stronger RCA decision based on multiple AI-generated perspectives.

**Module 8: Root Cause Fusion and Decision Module**

This module combines outputs from the RCA Agent, Validation Agent, and HolmesGPT Agent. It compares different explanations using evidence, confidence levels, and validation results. The system selects the most reliable root cause and prepares a final diagnostic response. This improves decision quality compared to traditional single-source analysis.

**Functionality:**

- Compares multiple RCA outputs.
- Evaluates supporting evidence.
- Calculates diagnostic confidence.
- Generates final RCA report.

**Outcome:**

A consolidated and trustworthy failure analysis result.

**Module 9: Human-in-the-Loop Governance Module**

This module introduces human supervision into the automated decision process. DevOps engineers can review AI-generated findings, approve remediation actions, reject suggestions, or provide additional feedback. Human decisions are captured to improve future system behaviour. This ensures safety and prevents uncontrolled automation.



**Functionality:**

- Displays RCA results to users.
- Allows approval or rejection.
- Captures expert feedback.
- Maintains operational control.

**Outcome:**

A safe AI-assisted DevOps system with transparency and human control.

**Module 10: Automated Remediation and Workflow Execution Module**

This module performs controlled execution of approved recovery actions. It connects AI recommendations with automation workflows to execute predefined remediation steps. Actions are controlled using policies to prevent unsafe modifications. The module converts AI decisions into practical infrastructure operations.

**Functionality:**

- Executes approved fixes.
- Triggers automation workflows.
- Restarts failed services.
- Applies configuration corrections.

**Outcome:**

Reduced manual effort and faster recovery from infrastructure failures.

**Module 11: Policy and Compliance Control Module**

This module ensures that automated actions follow predefined operational rules. It categorizes actions based on risk levels and determines whether approval is required. Sensitive infrastructure changes are restricted unless authorized by humans. This provides governance over AI-driven automation.

**Functionality:**

- Defines automation policies.
- Classifies actions.
- Blocks unsafe operations.
- Controls execution permissions.

**Outcome:**

A secure remediation mechanism suitable for enterprise environments.

**Module 12: Continuous Learning and Knowledge Update Module**

This module enables the system to improve from every resolved incident. After failure resolution, the final RCA, executed actions, and human feedback are stored as new organizational knowledge. Future incidents can use this experience for faster diagnosis. This creates a continuously evolving AIOps system.

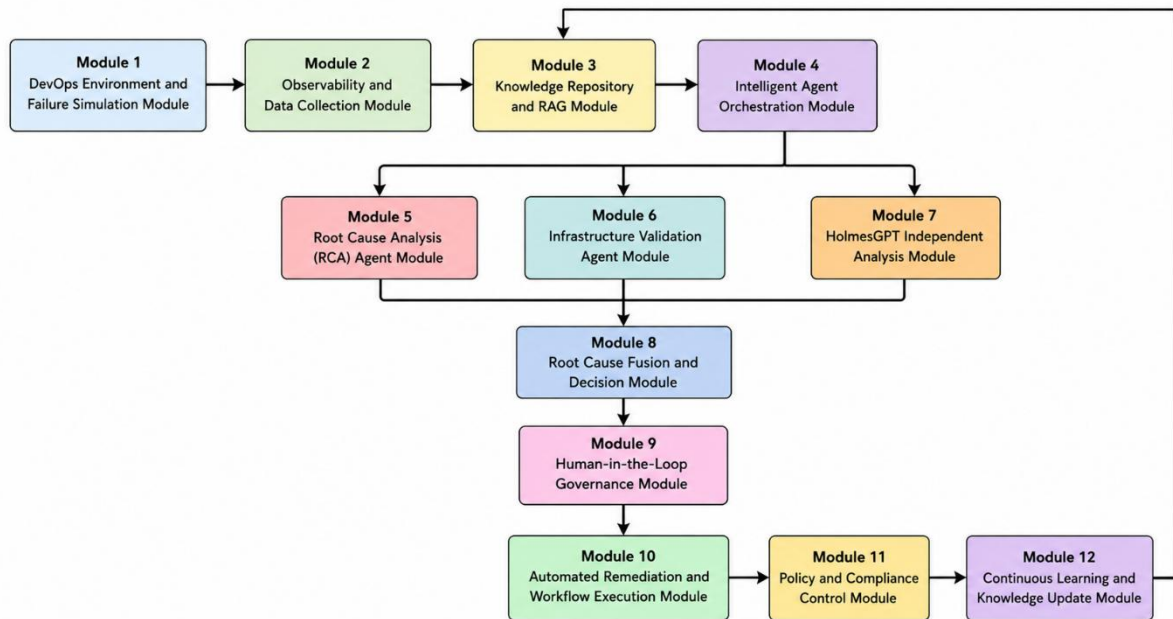
**Functionality:**

- Captures resolved incidents.
- Stores successful solutions.
- Updates knowledge repository.
- Improves future recommendations.

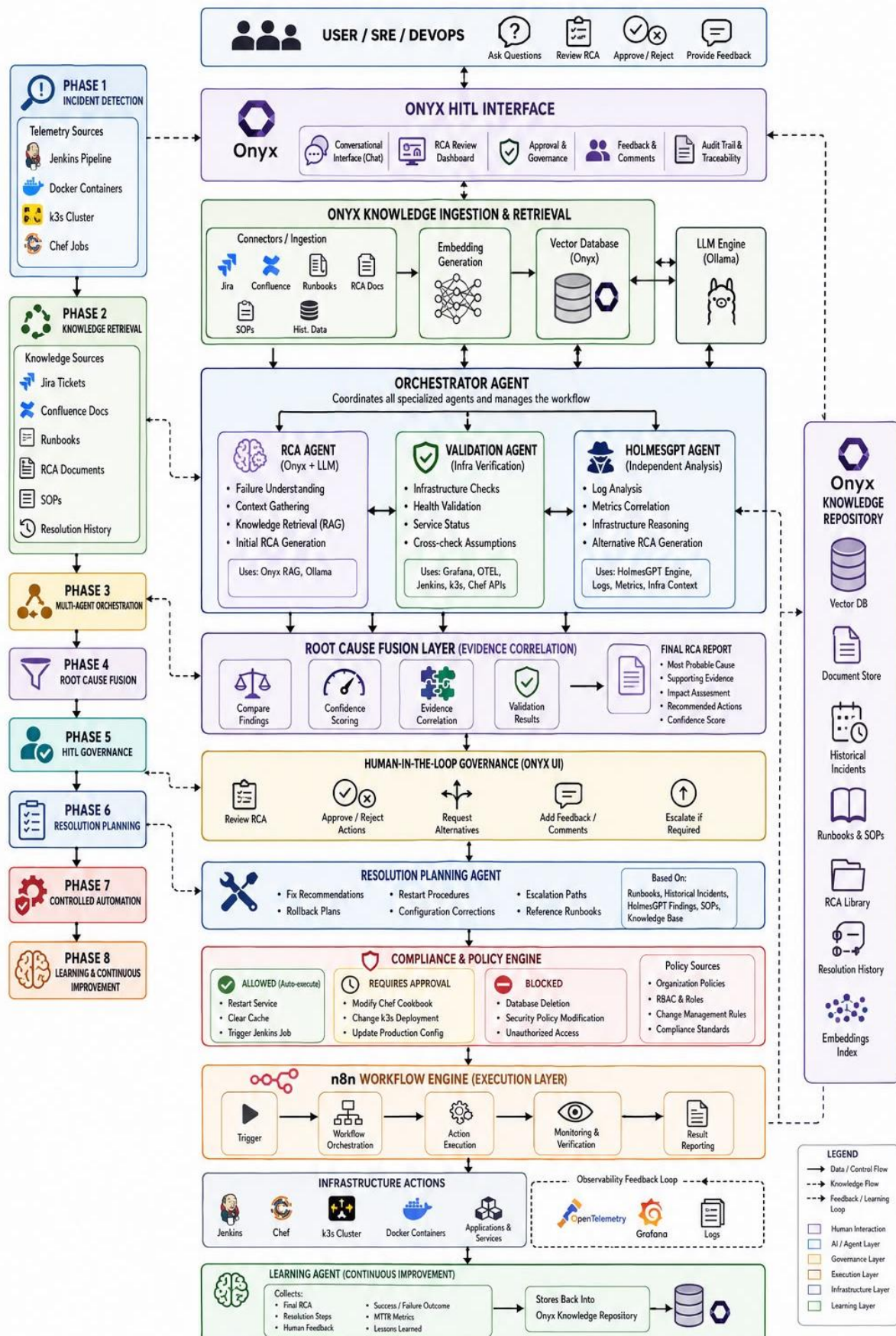
### Outcome:

A self-learning operational intelligence platform that becomes more accurate over time.

### Project Modules Diagram:



## 4. Functional Block Diagram



## Functional Description of the Project

### Phase 1 – Incident Detection

The Incident Detection phase is the starting point of the entire framework. In a DevOps environment, multiple infrastructure and application components continuously operate to deliver services. These components include Jenkins pipelines for software deployment, Docker containers for application execution, k3s clusters for container orchestration, and Chef for configuration management. During normal operation, these systems generate large amounts of telemetry data such as logs, metrics, traces, alerts, and system events.

When a failure occurs—such as a pipeline build failure, application crash, pod failure, configuration error, service outage, or abnormal resource utilization—the monitoring systems detect the anomaly. Tools like OpenTelemetry collect observability data, while Grafana visualizes metrics and generates alerts when predefined thresholds are violated. This phase identifies that a problem exists, gathers initial incident information, and forwards the incident to the next phase for detailed investigation. Essentially, this phase answers the question: "What failed and when did it fail?"

### Phase 2 – Knowledge Retrieval

Once an incident is detected, the framework must understand the context of the failure. Simply knowing that a service has failed is not enough; the system needs access to organizational knowledge that can help explain why the failure occurred.

This phase uses Onyx as the enterprise knowledge platform. Onyx connects to multiple knowledge sources such as Jira tickets, Confluence documentation, runbooks, RCA reports, SOPs, and historical resolution records. These documents contain valuable operational knowledge accumulated over time by engineers and support teams.

The documents are processed and converted into vector embeddings, allowing semantic search rather than simple keyword matching. When a new incident occurs, Onyx searches its knowledge repository to retrieve information related to similar past incidents, troubleshooting procedures, and documented fixes. Ollama provides the local LLM that helps interpret the retrieved information and prepare contextual knowledge for later reasoning stages.

This phase answers the question: "Have we seen something similar before, and what knowledge exists that can help us understand it?"

### Phase 3 – Multi-Agent Orchestration

This is the intelligence core of the entire framework. Rather than depending on a single AI model, the system employs multiple specialized agents, each performing a different investigative role. The Orchestrator Agent acts like a team leader that coordinates these agents and manages the flow of information among them.

The RCA Agent begins by examining the incident details and retrieved organizational knowledge. Using Onyx's RAG capabilities and the Ollama LLM, it analyzes failure symptoms and generates an initial hypothesis regarding the root cause.

At the same time, the Validation Agent independently verifies the actual state of the infrastructure. It checks system health, service availability, deployment status, configuration settings, cluster conditions, and monitoring data through APIs connected to Grafana, OpenTelemetry, Jenkins, k3s, and Chef. This helps determine whether the assumptions made by the RCA Agent are supported by real infrastructure evidence.

Simultaneously, HolmesGPT performs an independent analysis without relying on the RCA Agent's conclusions. It analyzes logs, correlates metrics, studies infrastructure behavior, and generates its own root cause hypothesis. Since HolmesGPT works independently, it acts as a second opinion and reduces the risk of AI hallucinations or incorrect assumptions.

The Orchestrator Agent collects findings from all agents and prepares them for comparison. This phase answers the question: "What are the possible causes of the incident according to different expert agents?"

## **Phase 4 – Root Cause Fusion Layer**

The Root Cause Fusion Layer is one of the most innovative parts of the architecture. Traditional systems typically trust the output of a single monitoring tool or AI model. However, relying on one source may lead to incorrect diagnoses.

In this framework, multiple independent sources of evidence are compared and correlated. The RCA Agent's findings, Validation Agent's infrastructure checks, and HolmesGPT's independent analysis are brought together into a common evidence evaluation layer.

The system compares supporting evidence, identifies agreements and disagreements between agents, and calculates confidence scores based on validation results. If multiple agents independently arrive at similar conclusions, confidence in the diagnosis increases. If conflicting conclusions exist, additional evidence is examined before producing the final assessment.

The output is a Final Root Cause Analysis Report containing the most probable root cause, supporting evidence, affected systems, business impact, confidence score, and recommended corrective actions.

This phase answers the question: "Which explanation is most likely to be correct based on all available evidence?"

## **Phase 5 – Human-in-the-Loop Governance**

Even though AI can perform sophisticated analysis, critical operational decisions should not be made without human oversight. This phase introduces Human-in-the-Loop (HITL) governance to ensure accountability, transparency, and trust.

The generated RCA report is presented to DevOps engineers, SREs, or system administrators through the Onyx interface. Users can review the diagnosis, inspect supporting

evidence, ask questions, request clarification, seek alternative recommendations, approve proposed actions, reject suggestions, or provide feedback.

For example, if the AI recommends restarting a production service, the engineer can review the reasoning before approving the action. If the engineer believes another approach is more suitable, they may reject the recommendation and provide corrective feedback.

This phase ensures that AI remains an assistant rather than an uncontrolled decision-maker. It answers the question: "Do human experts agree with the AI's findings and recommendations?"

## **Phase 6 – Resolution Planning**

Once the root cause has been verified and approved, the framework focuses on determining how the problem should be resolved. The Resolution Planning Agent uses information from runbooks, SOPs, historical incidents, organizational policies, and previous successful resolutions to generate a structured recovery plan.

Instead of simply suggesting a fix, the agent develops a complete remediation strategy. This may include restarting failed services, rolling back faulty deployments, correcting configuration errors, scaling infrastructure resources, updating dependencies, or escalating the issue to specialized teams.

The agent can also provide multiple recovery options ranked according to risk, complexity, and likelihood of success. Rollback procedures and contingency plans are included to ensure safe recovery if the primary solution fails.

This phase answers the question: "What is the safest and most effective way to resolve the incident?"

## **Phase 7 – Controlled Automation**

After a resolution plan has been approved, the framework moves into the execution phase. However, not every action should be executed automatically. Some actions may be low risk, while others may have serious consequences if performed incorrectly.

Before execution, every proposed action is evaluated by the Compliance and Policy Engine. The engine checks organizational policies, RBAC permissions, security requirements, compliance regulations, and change management guidelines.

Actions are classified into three categories:

### **Allowed Actions**

These are low-risk operations that can be executed automatically.

Examples:

- Restarting services
- Clearing caches
- Triggering Jenkins jobs

### **Approval Required Actions**

These actions may impact production systems and therefore require explicit human approval.

Examples:

- Updating Kubernetes deployments
- Modifying Chef configurations
- Changing production settings

### **Blocked Actions**

These actions violate organizational policies or present unacceptable risk.

Examples:

- Database deletion
- Unauthorized access attempts
- Security policy modifications

Approved actions are executed through the n8n Workflow Engine, which orchestrates workflows and interacts with infrastructure tools. The framework then monitors the execution results to verify whether the issue has been successfully resolved.

This phase answers the question: "How can the approved solution be executed safely and in compliance with organizational policies?"

## **Phase 8 – Learning and Continuous Improvement**

The final phase ensures that every incident becomes a learning opportunity for the system. After the incident is resolved, the Learning Agent collects information generated throughout the entire lifecycle of the incident.

This includes the final RCA report, remediation actions, execution outcomes, human approvals, engineer feedback, success or failure status, and operational metrics such as Mean Time to Resolution (MTTR).

The collected knowledge is processed and stored back into the Onyx Knowledge Repository. New RCA reports become future reference material, successful fixes become new runbook knowledge, and human feedback helps improve future recommendations.

As more incidents are processed, the framework gradually develops a richer operational memory and becomes increasingly accurate in diagnosing failures and recommending solutions. This creates a self-improving AIOps ecosystem.

This phase answers the question: "What did the system learn from this incident, and how can that knowledge improve future incident handling?"

## 5. Major Technical Specifications

| Tool / Component                      | Process                                    | Purpose                                                                               |
|---------------------------------------|--------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Jenkins</b>                        | Incident Detection, Validation, Automation | CI/CD pipeline monitoring, deployment verification, and automated job execution       |
| <b>Docker Containers</b>              | Incident Detection, Automation             | Hosts application services and provides container-level telemetry                     |
| <b>k3s Cluster</b>                    | Incident Detection, Validation, Automation | Kubernetes workload orchestration and health verification                             |
| <b>Chef</b>                           | Incident Detection, Validation, Automation | Configuration management and infrastructure provisioning                              |
| <b>OpenTelemetry (OTEL)</b>           | Detection, Validation, Monitoring          | Collects traces, metrics, and observability data                                      |
| <b>Grafana</b>                        | Detection, Validation, Monitoring          | Dashboard visualization, alerting, and infrastructure health monitoring               |
| <b>Logs</b>                           | Detection, HolmesGPT Analysis              | Failure investigation and evidence collection                                         |
| <b>Jira</b>                           | Knowledge Retrieval                        | Provides incident tickets and historical problem records                              |
| <b>Confluence</b>                     | Knowledge Retrieval                        | Stores technical documentation and troubleshooting knowledge                          |
| <b>Runbooks</b>                       | Knowledge Retrieval, Resolution Planning   | Provides standard recovery procedures                                                 |
| <b>SOP Documents</b>                  | Knowledge Retrieval                        | Supplies operational best practices and procedures                                    |
| <b>Historical Incident Repository</b> | Knowledge Retrieval, Learning              | Stores previous incidents and resolutions                                             |
| <b>Onyx</b>                           | Knowledge Retrieval, HITL Interface        | Enterprise RAG platform, document retrieval, chat interface, and governance dashboard |
| <b>Embedding Generator</b>            | Knowledge Retrieval                        | Converts documents into vector representations                                        |
| <b>Vector Database</b>                | Knowledge Retrieval                        | Stores embeddings for semantic search                                                 |
| <b>Ollama</b>                         | RCA Agent                                  | Provides local LLM inference for reasoning and RCA generation                         |
| <b>RCA Agent</b>                      | Multi-Agent Layer                          | Generates initial root cause analysis using RAG and LLM reasoning                     |



| <b>Tool / Component</b>               | <b>Process</b>         | <b>Purpose</b>                                                                     |
|---------------------------------------|------------------------|------------------------------------------------------------------------------------|
| <b>Validation Agent</b>               | Multi-Agent Layer      | Verifies infrastructure state and validates RCA assumptions                        |
| <b>HolmesGPT</b>                      | Multi-Agent Layer      | Independently performs log analysis, metrics correlation, and RCA generation       |
| <b>Orchestrator Agent</b>             | Multi-Agent Layer      | Coordinates all agents and manages workflow execution                              |
| <b>Root Cause Fusion Layer</b>        | RCA Fusion             | Correlates evidence and generates final RCA report                                 |
| <b>Onyx UI</b>                        | HITL Governance        | Allows review, approval, feedback, and RCA interaction                             |
| <b>Resolution Planning Agent</b>      | Resolution Planning    | Creates remediation and rollback strategies                                        |
| <b>Compliance &amp; Policy Engine</b> | Controlled Automation  | Enforces governance, security, and approval policies                               |
| <b>n8n Workflow Engine</b>            | Controlled Automation  | Executes approved remediation workflows                                            |
| <b>Applications &amp; Services</b>    | Automation Target      | Systems that receive corrective actions                                            |
| <b>Learning Agent</b>                 | Continuous Improvement | Captures lessons learned and updates knowledge repository                          |
| <b>Onyx Knowledge Repository</b>      | Learning & Retrieval   | Central storage for documents, incidents, embeddings, and organizational knowledge |

## 6. Design Considerations

- The framework was designed using a **multi-agent architecture** instead of a single AI agent to improve RCA accuracy through independent analysis, validation, and cross-verification of findings.
- **Human-in-the-Loop (HITL) governance** was incorporated to ensure that critical operational decisions remain under human control, reducing risks associated with fully autonomous actions.
- A **Retrieval-Augmented Generation (RAG)** approach was chosen to ground AI reasoning on organizational knowledge rather than relying solely on LLM-generated responses, thereby reducing hallucinations.
- The system leverages **local LLM execution through Ollama** to maintain data privacy, avoid external API dependencies, and enable operation within controlled environments.
- A **vector-based knowledge repository** was selected to support semantic search and contextual retrieval of historical incidents, runbooks, SOPs, and RCA documents.
- Independent validation through **HolmesGPT and infrastructure verification agents** was included to improve confidence in root cause identification and minimize false-positive diagnoses.
- A dedicated **Root Cause Fusion Layer** was introduced to combine multiple analysis outputs and generate a more reliable RCA than any individual agent could provide.
- **Policy-driven automation** was implemented to ensure remediation actions comply with organizational governance, security requirements, and change management practices.
- A continuous **feedback learning mechanism** was incorporated so that every resolved incident contributes new knowledge to future investigations and recommendations.
- **Open-source technologies** were preferred to reduce implementation costs, avoid vendor lock-in, and facilitate customization for research purposes.

### Implementation Consideration:

This project is implemented entirely in a local environment using open-source tools and frameworks. The DevOps infrastructure used for experimentation is a test environment containing randomly deployed projects and workloads for failure simulation. All agents, integrations, orchestration logic, knowledge repositories, and automation workflows have been created, configured, and executed locally on the system without dependence on external proprietary services.

## 7. Future Plan

| Phases                        | Start Date-End Date         | Work to be done                                                                 | Status      |
|-------------------------------|-----------------------------|---------------------------------------------------------------------------------|-------------|
| Dissertation Outline & Design | 10 May 2026 – 18 May 2026   | Literature Review, Tools Selection, Architectural Analysis and Design           | Completed   |
| Development                   | 25 May 2026 – 15 July 2026  | Development Activity                                                            | In Progress |
| Testing                       | 16 July 2026 – 24 July 2026 | Software Testing, User Evaluation & Conclusion                                  | Pending     |
| Dissertation Review           | 25 July 2026 - 2 Aug 2026   | Submit Dissertation to Supervisor & Additional Examiner for review and feedback | Pending     |
| Submission                    | 4 Aug 2026 - 10 Aug 2026    | Final Review and submission of Dissertation                                     | Pending     |

## 8. Abbreviations

| Abbreviation | Full Form                                      |
|--------------|------------------------------------------------|
| AI           | Artificial Intelligence                        |
| AIOps        | Artificial Intelligence for IT Operations      |
| API          | Application Programming Interface              |
| CI/CD        | Continuous Integration / Continuous Deployment |
| DevOps       | Development and Operations                     |
| HITL         | Human-in-the-Loop                              |
| k3s          | Lightweight Kubernetes Distribution            |
| LLM          | Large Language Model                           |
| MTTR         | Mean Time to Resolution                        |
| OTEL         | OpenTelemetry                                  |
| RAG          | Retrieval-Augmented Generation                 |
| RBAC         | Role-Based Access Control                      |
| RCA          | Root Cause Analysis                            |
| SOP          | Standard Operating Procedure                   |
| SRE          | Site Reliability Engineering                   |
| UI           | User Interface                                 |